

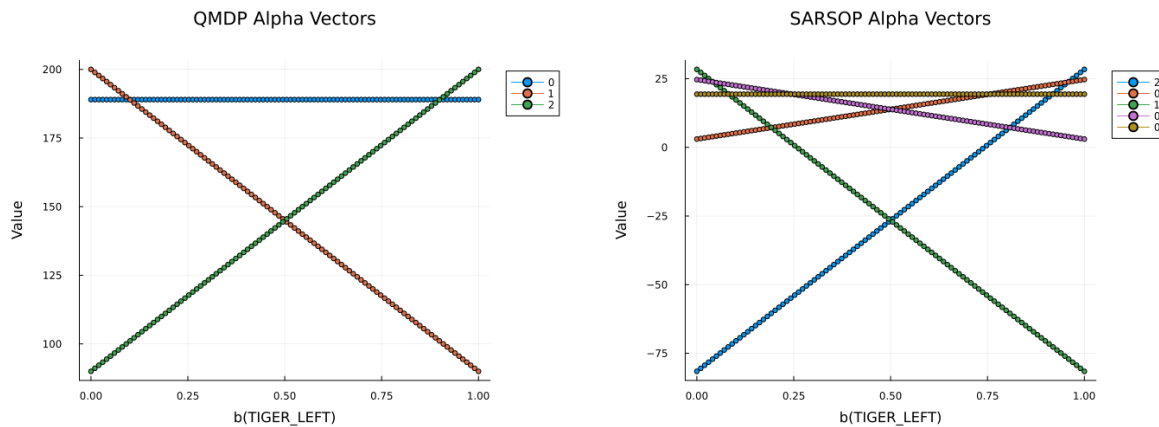
ASEN 5264 - DMU

Homework #6

Bijan Jourabchi

Problem 1:

a)

Figure 1: QMDP and SARSOP α -vector

b) For the QMDP policy, the average return is 19.4 with a SEM of 0.006. For SARSOP, we see very similar performance with an average return of 19.8 and SEM of 0.006.

Problem 2:

	QMDP	Heuristic	SARSOP
Average Return	65.32	73.01	78.18

Table 1: Average return over 1000 simulations for different policies

From these results it is clear that the cancer problem has a much larger gap in performance between QMDP and SARSOP compared to the tiger problem. I believe this is because the cancer problem is much less observable than the tiger problem. Since QMDP returns an optimistic policy there are regions of the belief space where it is overconfident in its action. This isn't as much of a problem in the tiger problem because it's much easier to observe the true state. This can be seen best in Figure 1, where there are regions of the belief space where QMDP is confident enough to open a door, and SARSOP is not, however these regions are small. In the cancer problem, I expect these regions are much larger and as a result QMDP performs significantly worse.

2.1 Code For Problem 1 and 2

```

1  using POMDPs
2  using LinearAlgebra: dot
3  using QMDP
4  using DMUStudent.HW6
5  using POMDPTools: transition_matrices, reward_vectors, SparseCat,
    Deterministic, RolloutSimulator, DiscreteBelief, FunctionPolicy,
    ordered_states, ordered_actions, DiscreteUpdater,
    has_consistent_distributions, Uniform
6  using QuickPOMDPs: QuickPOMDP
7  using POMDPModels: TigerPOMDP, TIGER_LEFT, TIGER_RIGHT, TIGER_LISTEN,
    TIGER_OPEN_LEFT, TIGER_OPEN_RIGHT
8  # using SARSOP: SARSOPSolver
9  using NativeSARSOP: SARSOPSolver
10 using Statistics: mean, std
11 using ProgressMeter
12
13 #####
14 # Problem 1: Tiger
15 #####
16
17 #-----
18 # Updater
19 #-----
20
21 struct HW6Updater{M<:POMDP} <: Updater
22     m::M
23 end
24
25 function POMDPs.update(up::HW6Updater, b::DiscreteBelief, a, o)
26     bp_vec = zeros(length(states(up.m)))
27
28     for sp in states(up.m)
29         bp_vec[stateindex(up.m,sp)] = Z(up.m,a,sp,o) * sum(s->T(up.m,s,a,
30             sp) * beliefvec(b)[stateindex(up.m,s)], states(up.m))
31     end
32
33     # Handle case where sum is 0 (no observation match)
34     total = sum(bp_vec)
35     if total > 0
36         bp_vec = bp_vec/total
37     else
38         # If no valid observation, return uniform belief
39         bp_vec = ones(length(states(up.m))) / length(states(up.m))
40     end
41
42     return DiscreteBelief(up.m, bp_vec)
43 end
44
45 # Note: you can access the transition and observation probabilities
46 # through the POMDPs.transition and POMDPs.observation, and query
47 # individual probabilities with the pdf function. For example if you want
48 # to use more mathematical-looking functions, you could use the

```

```

    following:
45 # Z(o | a, s') can be programmed with
46 Z(m::POMDP, a, sp, o) = pdf(observation(m, a, sp), o)
47 # T(s' | s, a) can be programmed with
48 T(m::POMDP, s, a, sp) = pdf(transition(m, s, a), sp)
49 # POMDPs.transition and POMDPs.observation return distribution objects. See
    the POMDPs.jl documentation for more details.
50
51 # This is needed to automatically turn any distribution into a discrete
    belief.
52 function POMDPs.initialize_belief(up::HW6Updater, distribution::Any)
53     b_vec = zeros(length(states(up.m)))
54     for s in states(up.m)
55         b_vec[stateindex(up.m, s)] = pdf(distribution, s)
56     end
57     return DiscreteBelief(up.m, b_vec)
58 end
59
60 # Note: to check your belief updater code, you can use POMDPTools:
    DiscreteUpdater. It should function exactly like your updater.
61 m = TigerPOMDP()
62 up = HW6Updater(m)
63 b0 = POMDPs.initialize_belief(up, Uniform(states(m)))
64
65 #-----
66 # Policy
67 #-----
68
69 struct HW6AlphaVectorPolicy{A} <: Policy
70     alphas::Vector{Vector{Float64}}
71     alpha_actions::Vector{A}
72 end
73
74 function POMDPs.action(p::HW6AlphaVectorPolicy, b::DiscreteBelief)
75
76     # Fill in code to choose action based on alpha vectors
77     return p.alpha_actions[argmax(a -> dot(p.alphas[a], beliefvec(b)), 1:
        length(p.alpha_actions))]
78 end
79
80 beliefvec(b::DiscreteBelief) = b.b # this function may be helpful to get
    the belief as a vector in stateindex order
81
82
83 #-----
84 # QMDP
85 #-----
86
87 function qmdp_solve(m, discount=discount(m))
88
89     # Fill in Value Iteration to compute the Q-values
90     A = actions(m)
91     R = reward_vectors(m)
92     T = transition_matrices(m)

```

```

93
94     S = states(m)
95
96     tol = 1e-6
97     n_states = length(S)
98     n_actions = length(A)
99
100
101     # Value fnx initialization
102     V = ones(n_states)
103     V_prev = zeros(n_states)
104
105     # State Value fnx
106     Q = zeros(n_states, n_actions)
107
108     while maximum(abs.(V-V_prev)) > tol
109
110         V = copy(V_prev)
111
112         for (i,a) in enumerate(A)
113             # R -> [S, 1], T-> [S x S], V -> [S x 1]
114             Q[:, i] = R[a] + discount * (T[a] * V)
115         end
116
117         V_prev = vec(maximum(Q, dims=2)) # Col wise max
118
119     end
120
121     acts = actiontype(m)[]
122
123     alphas = Vector{Float64}[]
124
125     for a in actions(m)
126
127         # Fill in pseudo alpha vector calculation
128         # Note that the ordering of the entries in the pseudo alpha
129         # vectors must be consistent with stateindex(m, s) (states(m)
130         # does not necessarily obey this order, but ordered_states(m)
131         # does.)
132         push!(acts, a)
133         alpha = [Q[stateindex(m,s), actionindex(m,a)] for s in S]
134         push!(alphas, alpha)
135     end
136
137     return HW6AlphaVectorPolicy(alphas, acts)
138 end
139
140 m = TigerPOMDP()
141 qmdp_p = qmdp_solve(m)
142 # Note: you can use the QMDP.jl package to verify that your QMDP alpha
143 # vectors are correct.
144 sarsop_p = solve(SARSOPSolver(), m)
145 up = HW6Updater(m)

```

```

143
144 MC_qmdp = [simulate(RolloutSimulator(max_steps=500), m, qmdp_p, up) for _
               in 1:5000]
145 MC_sarsop = [simulate(RolloutSimulator(max_steps=500), m, sarsop_p, up)
               for _ in 1:5000]
146 qmdp_mean = mean(MC_qmdp)
147 sarsop_mean = mean(MC_sarsop)
148
149 @show qmdp_mean
150 @show std(MC_qmdp)/5000
151 @show sarsop_mean
152 @show std(MC_sarsop)/5000
153
154 solver = QMDPSolver(max_iterations=200,
155                     belres=1e-6,
156                     verbose=false
157                     )
158
159 # run the solver
160 policy = solve(solver, m)
161
162 # @show qmdp_p.alphas
163 # @show policy.alphas
164
165 #-----
166 # Plot Alpha Vectors
167 #-----
168
169 using Plots
170
171 # where b(TIGER_RIGHT) = 1 - b(TIGER_LEFT)
172
173 # Generate belief points to evaluate
174 n_belief_points = 100
175 b_left = range(0, 1, length=n_belief_points)
176
177 # Evaluate QMDP alpha vectors
178 qmdp_plot = plot(
179     xlabel = "b(TIGER_LEFT)",
180     ylabel = "Value",
181     title = "QMDP Alpha Vectors",
182     legend = :outertopright,
183     lw = 2
184 )
185
186 for (i, alpha) in enumerate(qmdp_p.alphas)
187     values = [alpha[1] * bl + alpha[2] * (1 - bl) for bl in b_left]
188     action_label = "$(qmdp_p.alpha_actions[i])"
189     plot!(qmdp_plot, b_left, values, label = action_label, marker = :
           circle, ms = 3)
190 end
191
192 # Evaluate SARSOP alpha vectors
193 sarsop_plot = plot(

```

```

194     xlabel = "b(TIGER_LEFT)",
195     ylabel = "Value",
196     title = "SARSOP_Alpha_Vectors",
197     legend = :outertopright,
198     lw = 2
199 )
200
201 for (i, alpha) in enumerate(sarsop_p.alphas)
202     values = [alpha[1] * b1 + alpha[2] * (1 - b1) for b1 in b_left]
203     action_label = "$({sarsop_p.action_map[i]})"
204     plot!(sarsop_plot, b_left, values, label = action_label, marker = :
        circle, ms = 3)
205 end
206
207 # Combine into side-by-side layout
208 combined_plot = plot(qmdp_plot, sarsop_plot, layout = (1, 2), size =
    (1400, 500), margin = 10Plots.mm)
209 savefig(combined_plot, "alpha_vectors_comparison.png")
210
211 #####
212 # Problem 2: Cancer
213 #####
214
215 cancer = QuickPOMDP(
216     states = [:healthy, :in_situ, :invasive, :death],
217     actions = [:wait, :test, :treat],
218     observations = [true, false],
219     transition = function (s, a)
220         if s == :healthy
221             return SparseCat([:healthy, :in_situ], [0.98, 0.02])
222         elseif s == :in_situ
223             if a == :treat
224                 return SparseCat([:healthy, :in_situ], [0.6, 0.4])
225             else
226                 return SparseCat([:in_situ, :invasive], [0.9, 0.1])
227             end
228         elseif s == :invasive
229             if a == :treat
230                 return SparseCat([:healthy, :death, :invasive],
231                     [0.2, 0.2, 0.6])
232             else
233                 return SparseCat([:invasive, :death], [0.4, 0.6])
234             end
235         else
236             return Deterministic(:death)
237         end
238     end,
239     observation = function (a, sp)
240         if a == :test
241             if sp == :healthy
242                 return SparseCat([true, false], [0.05, 0.95])
243             elseif sp == :in_situ
244                 return SparseCat([true, false], [0.8, 0.2])
245             elseif sp == :invasive

```

```

246     return Deterministic(true)
247 end
248 elseif a == :treat
249     if sp in (:in_situ, :invasive)
250         return Deterministic(true)
251     end
252 end
253 return Deterministic(false)
254 end,
255 reward = function (s, a)
256     if s == :death
257
258         return 0.0
259     elseif a == :wait
260         return 1.0
261     elseif a == :test
262         return 0.8
263     elseif a == :treat
264         return 0.1
265     end
266 end,
267 discount = 0.99,
268 initialstate = Deterministic(:healthy),
269 isterminal = s->s==:death,
270 )
271
272 @assert has_consistent_distributions(cancer)
273
274 qmdp_p = qmdp_solve(cancer)
275 sarsop_p = solve(SARSOPSolver(), cancer)
276 up = HW6Updater(cancer)
277
278 heuristic = FunctionPolicy(function (b)
279
280     # Fill in your heuristic policy here
281     qmdp_action = POMDPs.action(qmdp_p, b)
282     qmdp_value = maximum(a -> dot(qmdp_p.
283         alphas[a], beliefvec(b)), 1:length(
284             qmdp_p.alphas))
285
286     values = [dot(qmdp_p.alphas[a], beliefvec(
287         b)) for a in 1:length(qmdp_p.alphas)]
288     sort!(values, rev=true)
289     uncertainty = abs(values[1] - values[2])
290
291     # Test more when uncertainty is high
292     threshold = 0.5
293     if uncertainty < threshold
294         return :test # Gather information
295             when uncertain
296     else
297         return qmdp_action # Follow QMDP when
298             confident
299     end

```

```

295         end
296     )
297
298
299 @show mean(simulate(RolloutSimulator(), cancer, qmdp_p, up) for _ in
        1:1000)      # Should be approximately 66
300 @show mean(simulate(RolloutSimulator(), cancer, heuristic, up) for _ in
        1:1000)
301 @show mean(simulate(RolloutSimulator(), cancer, sarsop_p, up) for _ in
        1:1000)      # Should be approximately 79

```

Problem 3:

- a) Yes, there is a DSE in this game. Player 1 and player 2 should always play P regardless of the other players choices.
- b) There is a single pure Nash equilibria of (P,P) in this game. Both players have a deterministic best response to play $a^i = P$.

c)

	P	F
P	0, 0	1, 1
F	1, 1	2, 2

- d) Yes, both players should always play F regardless of the other players actions.
- e) Yes, both players have a deterministic best response to play F. Thus the pure Nash equilibria is (F,F).
- f) No, for player 1, if player 2 plays P it's advantageous to also play P, however if player 2 plays F it is advantageous to play F. Thus, player 1 does not have a dominant strategy because their best response depends on what player 2 does.
- g) Yes, there are 2 pure Nash equilibria, (P,P) and (F,F) as well as a mixed Nash equilibrium. To compute the mixed Nash, we can begin by computing $\pi^1(s)$

$$U = 8\pi^1(P) + 9\pi^1(F)$$

$$U = 6\pi^1(P) + 10\pi^1(F)$$

$$1 = \pi^1(P) + \pi^1(F)$$

Solving this system of equations, we get the following result

$$\pi^1(P) = 1/3 \quad \pi^1(F) = 2/3$$

Since this game is symmetric, we get the same results for $\pi^2(s)$.

$$\pi^2(P) = 1/3 \quad \pi^2(F) = 2/3$$

Thus, each player should choose $a^i = P$ with probability 1/3, and choose $a^i = F$ with probability 2/3.

Problem 4:

To solve the Lasertag POMDP, I chose to use a POMCP implementation from BasicPOMCP.jl. For the most part I used the default POMCP values, and only modified the values of `tree_queries`, `max_depth`, and `c`. After tuning these values, I landed on `max_depth = 15`, `c = 10.0`, and `tree_queries = 1000`. However, I was able to achieve a score above 15 using only 50 tree queries. The reason I was able to get away with so few queries was due to my choice of a heuristic rollout policy. I chose to use a policy defined by QMDP as a rollout policy. This was the only way I was able to achieve a score over 15. I thought this was a good choice for a rollout policy since it was relatively lightweight computationally, and returned a good action consistently.

4.1 Code For Problem 4

```

1      using POMDPs
2  using QMDP
3  using DMUStudent.HW6
4  using POMDPTools: DiscreteBelief, DiscreteUpdater
5  using POMDPTools.Policies: FunctionPolicy, AlphaVectorPolicy
6  using BasicPOMCP
7
8  m = LaserTagPOMDP()
9
10  qsolver = QMDPSolver(
11      max_iterations=200,
12      belres=1e-6,
13      verbose=false
14  )
15
16  qmdp_policy = solve(qsolver, m)
17
18  function qmdp_state_action(m, qmdp_policy, s)
19      si = stateindex(m, s)
20      best_i = argmax(i -> qmdp_policy.alphas[i][si], eachindex(qmdp_policy.
21          action_map))
22      return qmdp_policy.action_map[best_i]
23  end
24
25  fo_qmdp_rollout = FunctionPolicy(s -> qmdp_state_action(m, qmdp_policy, s)
26      )
27
28  up = DiscreteUpdater(m)
29
30  function pomcp_solve(m, rollout_policy)
31      solver = POMCPSolver(
32          tree_queries=1000,
33          max_depth=15,
34          c=10.0,
35          default_action=first(actions(m)),
36          estimate_value=F0Rollout(rollout_policy)
37      )
38      return solve(solver, m)
39  end

```

```
38
39 pomcp_p = pomcp_solve(m, fo_qmdp_rollout)
40
41
42 @show HW6.evaluate((pomcp_p, up), "bijan.jourabchi@colorado.edu")
43 #@show HW6.evaluate((pomcp_p, up), n_episodes=100)
44 # When you get ready to submit, use this version with the full 1000
    episodes
45 # HW6.evaluate((qmdp_p, up), "REPLACE_WITH_YOUR_EMAIL@colorado.edu")
46
47 #-----
48 # Visualization
49 # (all code below is optional)
50 #-----
51
52 # You can make a gif showing what's going on like this:
53 #=using POMDPGifs
54 import Cairo, Fontconfig # needed to display properly
55
56 makegif(m, pomcp_p, up, max_steps=30, filename="lasertag.gif")
57
58 # You can render a single frame like this
59 using POMDPTools: stepthrough, render
60 using Compose: draw, PNG
61
62 history = []
63 for step in stepthrough(m, pomcp_p, up, max_steps=10)
64     push!(history, step)
65 end
66 displayable_object = render(m, last(history))
67 # display(displayable_object) # <-this will work in a jupyter notebook or
    if you have vs code or ElectronDisplay
68 draw(PNG("lasertag.png"), displayable_object) =#
```